



CMP 321 PROGRAMMING LANGUAGES

MIDTERM EXAM PAPER #1 – SAMPLE

Date: xxxx xx xxxxxxxx xxxx (1 hour)

Student ID: _____ **Student Name:** _____

Instructions: Write both your *name and ID* above. Read the questions carefully; write your answers clearly in the space provided. You can use draft paper, if necessary, but it must be returned with this paper. This is a *close book exam*: neither books or notes are allowed, nor is any communication with anyone but the examiner.

For Instructor's Use Only:

Q1	Programming languages	/ 10
Q2	Language features and costs	/ 10
Q3	Python syntax and features	/ 10
Q4	Python data structures	/ 10
Q5	Python objects and functions	/ 10

Total: / 50

Question 1: (a) Compare and rank the three generic versions of `max()` below in terms of *readability*, *writability*, and *reliability*, justifying briefly but precisely. (6 marks)

C: `#define max(a, b) ((a) > (b) ? (a) : (b))`

C++: `template<class T> const T& max (const T& a, const T& b)
{ return (a > b) ? a : b; }`

Python: `def max(a, b): return a if a > b else b`

	readability	writability	reliability
C			
C++			
Python			

(b) What does *orthogonality* refer to in a programming language? Explain clearly using as example the usage of the semicolon in C++. (4 marks)

A relatively small set of primitive constructs that can be combined in a relatively small number of ways (components independent of each other can be combined in different ways without having side effects to each other).

Necessary to add:-

```
int x = 5;
cout << "hello world!" << endl;
class myClass {};
```

Not Necessary to add:-

```
main() {}
void fun() {}
if () {}
for () {}
```

Thus, semicolons are not orthogonal.

Question 2: (a) How does *type inference* (as seen e.g. in Python) increase or decrease the various *costs* of a programming language? Discuss briefly *four* of them. (4 marks)

Training Programmers to use the language: Decreases the cost as programmers would have a reduced learning curve to learn the language.

Writing Programs: Decreases the cost. A programmer wouldn't have to write the data types for every variable. Plus, less code to write.

Compiling Programs: Increases the cost. It could catch type errors during compile-time.

Executing Programs: No real difference.

Language Implementations Systems: Increases the cost. Needed to be implemented by the developers of the feature.

Reliability: Increases the cost. It would be less readable but more writable. However, other programmers checking the code might have difficulty finding out the type of the variable and thus end up taking more time.

Maintaining Programs: Decreases the cost. You don't have to constantly check the type of a variable.

(b) Describe *three* key advantages of *scripting languages* such as Python. (3 marks)

1. They are typically interpreted at runtime by an interpreter (line-by-line).
2. If there is an error/bug, it would indicate it immediately and at what point.
3. Simpler syntax and higher level of abstraction.

(c) Explain briefly what a *hybrid* programming language is (vs. a compiled language, etc.) Is current *Java* hybrid then? Justify your answer. (3 marks)

A hybrid programming language is when there is a compromise between compilers and pure interpreters. They usually compile the code to an intermediate code (such as bytecode). This intermediate code is then interpreted on a user's computer/machine before running it.

No, Java is now fully compiler based. However, it used to be ~30 years ago.

Question 3: (a) Explain exactly what *dynamic typing* is, in Python, using the two lines of code shown on the right as example. (3 marks)

```
x = 1  
x = 'one'
```

x = 1 creates a reference to the int object (the type of variable x was int), but then x = 'one' causes the interpreter to create a string object (the type of variable x is now a string).

Variables are determined during runtime rather than compile-time.

(b) Indicate *what* each of the Python expressions hereafter represents and which feature is *unique* to Python i.e., does *not* exist in C++ and Java. (4 marks)

(i) '\u0078'

It is a Unicode section that represents some char, number, or special char assigned to that number. In this case, 'u0078' is lowercase x. Unique to Python.

(ii) [66, [],]

Represents a list containing "66", an empty list "[]", and a space " ". Unique to Python.

(iii) 1+7j

It's a complex number; 1 representing the real number, and 7j represents the imaginary number. Unique to Python.

(iv) N//2

Represents floor division; rounds down to the nearest integer. Unique to Python.

(c) Explain in detail what the following Python statement does *exactly* and what its output is: 'are scary movies worst at night?'[8:24:6] (3 marks)

It prints out 'yes'.

Start at index 8 (inclusive), end at 24 (exclusive), and jump by 6.

Question 4: (a) Consider the Python function below that replaces the elements from `alist` in the range `m..n` by the elements from the given `newlist` in the range `p..q`. Explain the limitations of the function and show an example that will not work. Rewrite it accordingly into a working, minimalist function. (4 marks)

```
def sub(alist, m, n, newlist, p, q):  
    for k in range(m,n):  
        alist[k] = newlist[p-m+k]
```

(b) Explain *why* the following Python code snippets are correct, or not. (2 marks)

```
zip = 26666 ; print('AUS P.O.', zip)  
def printPO(name='AUS', zip=26666): print(name,'P.O. :', zip)
```

(c) Write a Python-style Fibonacci function that implements the classic recurrence formula i.e., $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ with $\text{fib}(0) = 0$ and $\text{fib}(1) = 1$, while calculating each value *only once*. Hint: use a dictionary. (4 marks)

Question 5: (a) Explain what each line of the script below does *exactly*, including which objects are created, what the output is, if any, etc. (4 marks)

```
def f(n): return lambda x: x**n      # line 1
print( f(3) )                       # line 2
print( f(3)(4) )                    # line 3
```

(b) Explain briefly two “magic” function attributes of your choosing. (2 marks)

(c) Give the main *differences* between Python lambda expressions and functions. Accordingly, what is the intended use of lambda expressions in Python? Give two distinct usage examples. How is it different in Functional Programming languages, such as Haskell or Lisp? (4 marks)